

Quantumania — Computational Track Writeup

QSITE 2026 Quantum Coalition · Computational Track · Sep 25, 2026

Summary

Our `solve()` scores **67.0** across the six public benchmarks, against **283.5** for the provided baseline: **76% lower**. Every result passes the organisers' own scorer, and it runs in the default 60 s per benchmark.

The short version of how: we **search for routings using the real score**, we **spend most of our effort on choosing where qubits start**, and a **small learned policy** lets the search plan SWAPs a few gates ahead. Small circuits are also handed to an **exact solver**.

Benchmark	2Q gates	Baseline	Ours	SWAPs	Depth	Optimal?
ghz_star	7	14.0	6.5	2	9	proven
chain_trotter	9	15.0	4.5	0	9	proven
ladder_trotter	16	35.5	6.5	3	7	proven
qaoa_random	18	39.0	11.5	7	9	proven
dense_random	40	122.0	35.0	24	22	open
vqe_layers	45	58.0	3.0	0	6	proven
Total	135	283.5	67.0	36		5 of 6

`dense_random`'s 35.0 is our best verified routing; a single 60 s run usually returns 35.5. The stored routings can be re-checked with `python -m qroute.verify results/*.json`.

What makes this hard

Two rules shape everything. First, a two-qubit gate only runs on neighbouring qubits, and each qubit on this chip has at most three neighbours, so distant qubits need SWAPs to meet. Second, gates must stay in **exact program order**, even gates that share no qubits. That second rule blocks the usual trick of running whichever gates are ready, which is how standard routers such as SABRE get their parallelism. Since the score is $\text{SWAPs} + \frac{1}{2} \times \text{depth}$, a good routing needs **few SWAPs and SWAPs that run side by side**.

Three ideas behind the search

1 Score what the judges score

Each search state remembers, for every qubit, the last layer it was busy. With that clock, the search knows exactly how much depth a SWAP adds, so it optimises SWAPs *and* depth, not SWAP count alone.

2 Take whole steps

Adding one SWAP at a time wanders. Instead, for the next gate we pick a whole shortest path and a meeting point, and both qubits walk towards each other at once. Their SWAPs run in parallel, so a long-distance gate costs about half the depth, and every step executes one gate.

3 Start in the right place

Once the starting placement is fixed, better routing barely helps: beam widths from 50 to 1,200 give the same score. So most of the time budget goes into searching for good starting placements.

How `solve()` works, step by step

1. **Try zero SWAPs.** If the circuit's interaction graph fits inside the chip (a subgraph match), we are done. This alone solves `chain_trotter` and `vqe_layers`.

2. **Search placements.** Build about 400 varied starting placements, score each with a quick, narrow search, then improve the best ones by local search: swap or move qubits within two hops, and shake things up when stuck.
3. **Polish.** Re-route the top placements under many search settings (216 combinations). The search is not monotone in its settings, so a different setting often finds a cheaper route from the same start.
4. **Plan ahead with the learned policy** (next section), then refine with forward-backward routing.
5. **Solve small circuits exactly.** Up to 24 gates, an exact CP-SAT model takes over (see *Exact solver*).
6. **Check and return.** Every candidate is scored by the organisers' validator before it can win, and the provided baseline is kept as a fallback, so `solve()` never returns an invalid routing.

The learned policy: planning SWAPs ahead

The problem it solves. The moves above only move qubits for the *current* gate. Often it pays to start moving a qubit earlier, on idle qubits, for a gate that is coming up. Allowing such "pre-positioning" SWAPs for any of the next four gates gives the search 5–6× more options, and our hand-written heuristic handles that badly: it overrates these moves and they crowd out good ones.

What the policy does. A small graph transformer (2.2M parameters) looks at the chip, the current placement and the next few gates, and scores every candidate move in one batch. Each search state keeps all of its ordinary moves plus the policy's four best pre-positioning moves, and the same network ranks the whole beam. It is trained to put sibling moves (alternatives from the same state) in the right order, since that is the only decision the search makes.

What made it work: better labels, not more data. Our first training round (1.2M states) labelled each move by what a plain search could make of it, and it only tied the solver without the policy. The second round re-labelled 144k states using the policy's own wider search, so a pre-positioning move is judged by what *further* pre-positioning can make of it. That moved the score. Training on anywhere from 12.5% to 100% of the data made no difference, and a third round tied the second.

Forward-backward refinement. SABRE's trick is to route the circuit, then route it backwards starting from where the qubits ended up, and repeat. It did nothing with our heuristic, but with the policy it improves every top placement. On `dense_random` it won 13 of 16 seeds (1 loss, 2 ties), taking the mean from 37.50 to 36.22.

Exact solver

For circuits up to 24 gates, a CP-SAT model places and routes with complete freedom: any SWAP, on any edge, in any layer. It finds trade-offs our step-by-step search cannot express. On `qaoa_random` it traded one extra SWAP for three fewer layers: 6 SWAPs at depth 12 (12.0) became 7 SWAPs at depth 9 (11.5).

Encoding the order rule. The strict gate order is the hard part to model. Our fix: each qubit carries a "watermark", the latest program gate that has influenced it so far. Watermarks travel with SWAPs, and gate k may only run on qubits whose watermarks are still below k . That is exactly the scorer's rule, so every solution the model finds is valid, and every valid routing is something the model can find.

Why five scores cannot improve. `chain_trotter` and `vqe_layers` use zero SWAPs and hit the minimum depth. `ghz_star` has one qubit that must meet seven partners on a chip where no qubit has more than three neighbours, which forces a minimum cost that we match. For `ladder_trotter` and `qaoa_random`, the exact model checked every possible depth and found no better score.

Making it reliable

Best of four, in parallel. One run leaves most of a 24-core machine idle. For circuits over 24 gates, `solve()` runs four independent searches on 6 cores each and keeps the best. Splitting the cores costs each run nothing, and the best of four is both better and steadier.

<code>dense_random</code> at 60 s	Mean	Worst
One run, all cores (16 seeds)	36.22	38.5
Best of 4 in parallel (8 trials)	35.56	36.0

Without a GPU, the policy is about 30× slower, so the learned stages switch off and their time goes to polishing. `dense_random` then averages 37.8 and finishes in 56 s (leaving the policy on would give 39.5 at about 62 s). **With more time,** budgets over 120 s run independent 60 s restarts (35.67 mean at 300 s, against 37.0 for one long run). **On unseen inputs** (18

extra programs with single-qubit gates, gaps in the qubit labels, all 20 qubits, up to 150 gates), all 36 runs, with and without a GPU, were valid and on time.

Compared with Qiskit's SABRE

Across 2,000 seeds per benchmark, SABRE's valid routings total **76.0** against our **67.0**. We tie on the four easy benchmarks and lead on qaoa_random and dense_random. SABRE reorders independent gates, which this challenge forbids, so only 2 of its 2,000 dense_random routings passed the scorer.

What didn't work, and what's open

- **More routing search.** On the best dense_random placement, a search 256× wider (width 4,096 vs 16) finds the same 35.0. There was nothing left for tree search to find.
- **Bigger networks.** An 18M-parameter net, a 2.2M one and even a random value function tied on three of the four routed benchmarks. What mattered was the move set and the labels, not model size.
- **Re-solving around a good solution** exactly (tube and window search) never improved the policy's routings.
- **Choosing placements with the policy.** It ranks placements differently from the heuristic (rank correlation 0.40), but re-ranking within 60 s cost more time than it gained.

Open: dense_random, at 35.0 best and 35.5 typical. Its lower bound (10.0) is too loose to tell how much of that gap is real; tighter lower bounds for dense circuits are the natural next step.